

UD03DOC01

Gestión de sistemas de directorios y archivos por comando de Linux

Índice

1. Gestión de sistemas de directorios y archivos por comando de Linux.....	2
1.1. Comando ayuda (man y help).....	2
1.1.1 man.....	2
1.1.2 help.....	3
1.1.3. Diferencia entre man y help.....	3
1.2. ¿Dónde estamos? (pwd).....	3
1.3. Listado de contenido de directorios (ls).....	3
1.3.1 Concatenar opciones.....	5
1.4. Tipos de ficheros.....	5
1.5 Enlaces (ln).....	6
1.6. Cambiar de directorios (cd).....	7
1.6.1 Rutas absolutas.....	8
1.6.2 Rutas relativas.....	8
1.7. Creación directorios (mkdir).....	8
1.8. Eliminación de directorios (rmdir).....	10
1.9. Copia de archivos (cp).....	10
1.10. Renombrar o mover un archivos (mv).....	11
1.11. Creación de ficheros (touch y cat).....	12
1.11.1 Touch.....	12
1.11.2 cat.....	12
1.12. Eliminación de ficheros (rm).....	12
1.13. Mostrar directorios como árbol (tree).....	13

1. Gestión de sistemas de directorios y archivos por comando de Linux

En entornos domésticos o de oficina se emplea la interfaz gráfica para el manejo y gestión de archivos. No obstante, los administradores de sistemas suelen hacer uso de la interfaz por línea de comandos, ya que su versatilidad y potencia de uso la convierte en una herramienta ideal.

Los comandos de Linux siguen una sintaxis:

`comando [opciones] [argumentos]`

Cada comando varía el tipo de opciones y argumentos que pueda utilizar, e incluso puede no utilizarse ninguno en caso de ser opcionales. Las opciones pueden ser cortas (una sola letra) o largas (una palabra), anteceditas por uno o dos guiones. Se pueden emplear ambos tipos de opciones con un mismo comando, aunque las opciones cortas pueden unirse. Tanto los comandos como las opciones y los argumentos que se usen son sensibles a mayúsculas y minúsculas.

1.1. Comando ayuda (man y help)

El comando para ver la ayuda en general o de un comando en concreto

1.1.1 man

Este comando se utiliza para conocer la documentación o aprender de un comando en especial. “man” de manual.

Por ejemplo si queremos conocer como se usa y qué hace `chmod` haríamos:

`man chmod`

1.1.2 help

Puedes utilizar el comando `help` en cualquier momento para obtener más información sobre el uso y la sintaxis de un comando específico.

Por ejemplo, queremos saber sobre el comando `cd`:

`help cd`

Tiene una serie de opciones

-d: Descripción breve de salida para cada tema. → `help -d cd`

-m: Mostrar el uso en formato de pseudo-página de manual. → `help -m cd`

-s: Solo muestra una breve sinopsis de uso para cada tema coincidente.

1.1.3. Diferencia entre man y help

El comando `man` es mucho más extenso, ya que es la definición del manual. La información de `help` es una ayuda rápida y concisa.

1.2. ¿Dónde estamos? (pwd)

Es lo que nos vamos a preguntar muchas veces, ¿en qué carpeta o directorio estamos?

El comando para averigüarlo es: **pwd**.

Es el nombre nemotécnico de “**P**rint **W**orking **D**irectory”. Y es justo lo que hace nos imprime el directorio actual donde estamos.

Opciones

-L: aunque en el intérprete de comandos se haya fijado la opción “physical”, si hay un enlace simbólico, este no se resuelve

-P: *con esta opción se resuelve un enlace simbólico*

1.3. Listado de contenido de directorios (ls)

Uno de los comandos que más se emplea es ls (list).

ls [opciones] [ficheros]

Permite listar el contenido de un directorio e información de archivos. Sus opciones más utilizadas son las siguientes:

- ✓ **-l: muestra en formato largo.**
- ✓ **-a:** muestra los archivos ocultos. En Linux los archivos ocultos son aquellos que empiezan con “.”. Si no indicamos esta opción, el comando ls no listará los archivos ocultos.
- ✓ **-h:** muestra el tamaño de cada fichero en K, M, G, etc.
- ✓ **t:** ordena por fecha de modificación.
- ✓ **-S:** lista los archivos ordenados por tamaño.
- ✓ **-r:** invierte el orden de salida.
- ✓ **-R:** lista recursivamente el contenido de cada directorio.
- ✓ **-i:** muestra el número de i-nodo.
- ✓ **-size:** muestra el tamaño de cada fichero en bloques.

Por defecto y sin argumentos, lista el directorio actual ordenado alfabéticamente. Si se indica más de uno, se listará el contenido de cada uno de ellos.

Cuando se emplea la opción “**-l**”, aparecen clasificadas en columnas la información propia de cada fichero listado:

Columna	Nombre	Descripción
1	Permisos y tipos	Muestra el tipo de archivo y sus permisos. El primer carácter indica el tipo - para archivo normal d para directorio l para enlace simbólico Le siguen nueve caracteres que representan los permisos de lectura (r), escritura (w) y ejecución (x) para el propietario, el grupo y otros, respectivamente.
2	Enlace físico	El número de enlaces físicos (hard links) que apuntan a ese archivo o directorio.
3	Propietario	El nombre del usuario propietario del archivo o directorio.
4	Grupo	El nombre del grupo propietario al que pertenece el archivo o directorio.
6	Tamaño	El tamaño del archivo en bytes (a menos que se use la opción -h para un formato legible por humanos, como KB, MB).
6	Fecha/hora de última modificación	La fecha y hora de la última modificación del archivo.
7	Nombre	El nombre del archivo o directorio.

Por ejemplo una salida podría ser:

```
-rw-r--r-- 1 usuario grupo 1024 nov 15 18:24 archivo.txt
```

En este ejemplo:

- -rw-r--r--: Es un archivo normal (-), con permisos de lectura/escritura para el propietario (rw-), y solo lectura para el grupo (r--) y otros (r--).
- 1: Tiene un enlace físico.
- usuario: El propietario es "usuario".
- grupo: El grupo propietario es "grupo".
- 1024: El tamaño es 1024 bytes.
- nov 15 18:24: Última modificación el 15 de noviembre a las 18:24.
- archivo.txt: El nombre del archivo es "archivo.txt".

Cualquier archivo o directorio se localiza e identifica dentro del árbol de directorios gracias a su ruta. Por ello, no pueden existir dos archivos con el mismo nombre en un mismo directorio.

Además, todos los directorios disponen, al menos, de dos entradas ocultas:

“.” el directorio actual

“..” el directorio padre.

Estas se definen automáticamente cuando se crea un directorio. El directorio actual hace referencia a él mismo y se emplea muy a menudo cuando se trabaja con rutas relativas, y también lo

hacen por defecto multitud de comandos. También el directorio padre se utiliza necesariamente para poder moverse por el árbol de directorios.

1.3.1 Concatenar opciones

Podemos usar varias opciones a la vez para un único comando.

Por ejemplo: quiero mostrar archivos y directorios en formato largo y los ocultos también.

```
ls -l -a
```

Pero lo más común es unir las opciones, así:

```
ls -la
```

1.4. Tipos de ficheros.

Los tipos de ficheros en Linux son muy importantes. Cualquier elemento físico (discos, impresoras, etc.) o lógico (directorio, enlace, etc.) se representa en Linux mediante un archivo, lo que facilita y estandariza la gestión de todos ellos. Se distinguen cuatro tipos elementales de ficheros:

- a) Regulares. Son ficheros ordinarios que contienen información de diversa naturaleza. Dentro de ellos se incluyen los ficheros ejecutables como un tipo de fichero regular que tiene activo algún permiso de ejecución y contienen código ejecutable.
- b) Directorios. Almacenan en su bloque de datos el número de i-nodo y el nombre de los archivos que contiene. Cuando son listados con el comando `ls` muestra la información accediendo a la estructura interna de cada i-nodo.
- c) Enlaces.
- d) Dispositivos. Son archivos que representan a dispositivos físicos. En el directorio `/dev` se localiza la mayoría de ellos. Se distinguen dos tipos de archivos de dispositivos:
 - Dispositivos por caracteres: suelen ser aquellos que no disponen de sistema de archivos, como impresoras, teclados, terminales de texto, etc. Transfieren los datos carácter a carácter.
 - Dispositivos por bloques: almacenan la información en bloques de datos físicamente, como, por ejemplo: discos duros, cintas magnéticas, unidades flash, etc.

Además, existen dispositivos virtuales, los cuales tienen un tratamiento especial, como `/dev/null`, también conocido como cubeta de bits, puesto que se suele emplear para eliminar o descartar toda información que sea enviada a él.

1.5 Enlaces (ln)

Existen dos tipos de enlaces:

- **Enlaces duros.** Los enlaces duros están estrechamente relacionados con un sistema especial de contabilidad interna. Cada enlace duro hace referencia a un llamado “inodo” y se le asigna un número de inodo inequívoco e inconfundible que también está asociado al archivo original. Un archivo solo se borra definitivamente en la administración de inodos, y por tanto también en el sistema, cuando todas las entradas (es decir, las referencias a este fichero) se han declarado inválidas al eliminarse y se ha puesto a cero el contador de enlaces interno.

Los inodos son estructuras de datos definidas que describen a un archivo único, contienen información de metadatos sobre él (pertenencia a un grupo, propietario, derechos de acceso, etc.) y documentan su ubicación de almacenamiento (en forma de una dirección de almacenamiento).

Se explican mejor con un ejemplo concreto. Un archivo de vídeo al que solo se puede acceder desde el directorio “Mis vídeos” puede cargarse también a partir del directorio “Mis vídeos-Copia de seguridad” creando un enlace duro. Si ahora se elimina el archivo original en “Mis vídeos” (en sentido puramente estricto, se borra la referencia primaria al archivo de vídeo), es posible seguir accediendo a este mediante la ruta paralela equivalente sin problema alguno (la ruta del enlace duro al archivo del directorio “Mis vídeos-Copia de seguridad”). Los enlaces duros tienen la ventaja de que una nota adicional en la administración del espacio no ocupa el doble de espacio.

Para crear enlaces duros, se emplea el comando `ln`, y su sintaxis es la siguiente:

ln fichero fichero_enlace.

Por ejemplo, crear un enlace duro llamado “enlace_duro_a_archivo_de_video” del video.mp4 en la misma carpeta del vídeo:

ln /home/peter/videos/video.mp4 enlace_duro_a_archivo_de_video

Otro ejemplo, crear un enlace duro llamado “enlace_duro_a_archivo_de_video” del video.mp4 en el escritorio:

ln /home/peter/videos/video.mp4 /home/peter/desktop/enlace_duro_a_archivo_de_video

- **Enlaces simbólicos o enlaces blandos.** El directorio que contiene un enlace simbólico almacena un i-nodo (diferente al i-nodo con el archivo que enlaza) y un nombre de fichero. Para acceder al fichero con el que enlaza, el i-nodo del enlace simbólico almacena en un campo la ruta de acceso al archivo destino, si esta es inferior a 60 bytes y, si es superior, se almacenará en el bloque de información de un extent. De esta manera, pueden referenciar archivos de otros sistemas de archivos, particiones y equipos en red.

Para crear enlaces simbólicos, se debe incluir la opción “-s” con ln: ln -s fichero fichero_enlace.

Por ejemplo, para que el enlace simbólico se encuentra en la misma carpeta:

```
ln -s /home/peter/video.mp4 enlace_simbólico_a_archivo_de_video
```

Otro ejemplo, podemos crear enlaces simbólicos en otros directorios, p. ej. en el escritorio:

```
ln -s /home/peter/videos/video.mp4 /home/peter/Desktop/enlace_simbólico_a_archivo_de_video
```

La propia orden ls dispone de la opción “-color”, que se encuentra activada por defecto en Ubuntu y permite discriminar el tipo de archivo según el color:

- Blanco: archivo regular.
- Verde: archivo ejecutable.
- Azul: directorio.
- Cian: enlace simbólico.
- Rojo: enlace roto

1.6. Cambiar de directorios (cd)

Este comando se utiliza para cambiar de directorio, hacia arriba del árbol de directorios o hacia abajo del árbol de directorios.

La sintaxis es: cd [opción] [directorio]

Veamos algunas opciones útiles:

Opción	Comando	Descripción
~	cd ~	Cambia directamente a tu directorio principal.
/	cd /	Cambia al directorio raíz.
-	cd -	Devuelve a tu directorio de trabajo inmediatamente anterior.
.	cd .	Se utiliza para hacer referencia el directorio actual.
..	cd ..	Se utiliza para salir del directorio una posición hacia arriba del árbol de directorios. Puedo ir varios pasos si concateno ../../../ en este caso voy hacia atrás 3 ramas.
ruta	cd /usr/local	Cambiar al subdirectorio /usr/local.

El símbolo ~ se obtiene dejando pulsada la tecla Shift y teclear el número 126.

1.6.1 Rutas absolutas

Una ruta absoluta define la ubicación de un directorio o fichero **teniendo en cuenta todo el árbol de directorios**, la cual comienza siempre en slash (/) o en la raíz.

Por ejemplo: `cd /home/usuario_76/universidad/universidad_2`

1.6.2 Rutas relativas

Definen la posición de un directorio o fichero **a partir de su ubicación actual**, puede comenzar con un punto(.), o con punto punto(..).

Por ejemplo:

`root@:/home/usuario1/trabajos/textos# cd ../../documentos`

Nos devuelve a

`root@:/home/usuario1/documentos`

1.7. Creación directorios (mkdir)

Se pueden crear directorios mediante la orden mkdir. Su sintaxis es la siguiente:

`mkdir lista_de_directorios`

Ejemplos:

1.- Crea el directorio scripts:

`mkdir scripts`

2.- Crea los directorios Contactos y Material en el mismo directorio de trabajo:

`mkdir Contactos Material`

3.- Crea estructuras:

Contactos

•**Cientes**

•Pedidos

•Facturas

•**Proveedores**

•Pedidos

•Facturas

`mkdir -p Contactos / {Clientes, Proveedores} / {Pedidos, Facturas}`

Opciones:

Opción	Comando	Descripción
-m o -mode	<code>mkdir -m 755 compartido</code>	Permite asignar determinados derechos al nuevo directorio. Los derechos se indican a continuación de la opción. El ejemplo crea el directorio compartido con los permisos <code>rw-r-xr-x</code> (lectura, escritura y ejecución para el propietario, y solo lectura y ejecución para el grupo y otros).
-p o -parents	<code>mkdir -p proyectos/frontend/html</code>	Permite crear el directorio o los directorios si todavía no existen. Ya que, si resulta que ya existe un directorio con el nombre y ubicación indicado en el comando, el sistema continúa sin mensaje de error. El ejemplo creará <code>proyectos</code> , <code>proyectos/frontend</code> y <code>proyectos/frontend/html</code> si no existen previamente.
-v o -verbose	<code>mkdir -v nuevo_directorio</code>	Muestra en la línea de comandos lo que <code>mkdir</code> está haciendo en ese momento. En la mayoría de los casos se utiliza junto con la opción <code>-p</code> . El ejemplo mostrará un mensaje como "se creó el directorio 'nuevo_directorio'".

Combinaciones:

```
mkdir -pv proyectos/backend/{api,web}
```

Crea el directorio padre `proyectos`, luego `proyectos/backend`, y dentro de este último crea dos directorios: `api` y `web`, mostrando un mensaje de confirmación por cada uno.

1.8. Eliminación de directorios (`rmdir`)

La eliminación de directorios se puede efectuar mediante la orden `rm -r`, si dispone de contenido, o empleando el comando `rmdir`, si el directorio se encuentra vacío.

```
rm lista_de_directorios
```

Ejemplos:

elimina el directorio vacío scripts: `rmdir scripts`:

Opciones:

Opción	Comando	Descripción
-d	<code>rmdir -d /home/documents</code>	Elimina los directorios vacíos
-r	<code>rmdir -r /home/documents</code>	Elimina directorios no vacíos y sus contenidos.
-f	<code>rmdir -f /home/documents</code>	Ignora cualquier aviso al eliminar un archivo protegido contra escritura.
-i	<code>rmdir -i /home/documents</code>	Emite un aviso antes de borrar cada archivo.
-p	<code>rmdir -p /home/documents/SSF</code>	Elimina un subdirectorio vacío y su directorio padre. Por ejemplo si SSF y documents están vacíos se borrarían.

Combinación:

Elimina, pidiendo confirmación, todos los archivos del directorio trabajo y el propio directorio.

`rm -ri ./trabajo`:

1.9. Copia de archivos (cp)

El comando utilizado para copiar la información de un archivo es `cp`. Su sintaxis es:

`cp [opciones] lista_archivos_origen destino`

Por defecto se sobrescribirá en el destino la lista de archivos de origen, en caso de equivalencia en los nombres.

Opción	Comando	Descripción
-i	<code>cp -i mi_directorio /ruta/de/destino</code>	Pregunta al usuario antes de sobrescribir un archivo de destino que ya existe.
-v o -verbose	<code>cp -v mi_directorio /ruta/de/destino</code>	Modo detallado (verbose). Muestra en la terminal cada archivo y directorio que se está copiando.
-r o -R	<code>cp -r mi_directorio /ruta/de/destino</code>	Copia directorios de forma recursiva, incluyendo todos sus subdirectorios y archivos.
-u	<code>cp -u mi_directorio /ruta/de/destino</code>	Actualiza. Copia el archivo solo si es más reciente que el archivo de destino o si el archivo de destino no existe.
-n	<code>cp -n mi_directorio /ruta/de/destino</code>	No sobrescribe ningún archivo existente, incluso si se usó la opción -i.

Ejemplos:

- `cp -i /etc/passwd ./passwd.bck`: copia el archivo `passwd`, solicitando confirmación, a otra ubicación (desde `/etc.` al directorio actual) cambiando además de nombre.
- `cp -r . /backup/`: copia el directorio actual a `/backup/` directorios completos.

1.10. Renombrar o mover un archivos (mv)

Se emplea la orden `mv` para mover o renombrar archivos. Su funcionamiento es similar a la orden `cp`, salvo que los archivos de origen desaparecen:

`mv [opciones] lista_archivos_origen destino`

Sus opciones más utilizadas son:

Opción	Comando	Descripción
-i	<code>mv -i archivo_viejo.txt /ruta/de/destino</code>	Pide confirmación antes de sobrescribir un archivo existente en el destino.
-v o -verbose	<code>mv -v archivo_viejo.txt /ruta/de/destino</code>	Muestra el nombre de cada archivo a medida que se mueve.
-f	<code>mv -f archivo_viejo.txt /ruta/de/destino</code>	Sobrescribe los archivos de destino sin pedir confirmación.
-u	<code>mv -u archivo_viejo.txt /ruta/de/destino</code>	Mueve el archivo o directorio solo si el archivo de origen es más reciente que el archivo de destino, o si el archivo de destino no existe.

1.11. Creación de ficheros (touch y cat)

1.11.1 Touch

Se usa principalmente para crear archivos vacíos o para actualizar la fecha y hora de acceso/modificación de archivos existentes. Se utiliza para crear archivos nuevos escribiendo `touch nombre_archivo`, y, en caso de que exista, para actualizar la marca de tiempo de un archivo existente escribiendo `touch nombre_archivo`.

Algunas opciones son:

Opción	Comando	Descripción
-c	<code>touch -c mi_archivo_existente.txt</code>	Evita crear un archivo si no existe.
-m	<code>touch -m mi_archivo.txt</code>	Modifica solo la fecha de modificación.
-d	<code>touch -d "2024-01-01 10:30" mi_archivo.txt</code>	Usa una cadena de fecha especificada en lugar de la hora actual.

1.11.2 cat

Se utiliza principalmente para ver el contenido de archivos de texto y combinar varios archivos en uno solo.

Si el archivo no existe lo crea, si ella existe muestra su contenido.

Algunas opciones son:

Opción	Comando	Descripción
-n	cat -n mi_archivo_existente.txt	Numera todas las líneas de la salida del archivo.
-b	cat -b mi_archivo.txt	Numera solo las líneas que no están vacías (ignora las líneas en blanco).
-s	cat -s mi_archivo.txt	Elimina las líneas en blanco repetidas y consecutivas, mostrando solo una línea en blanco en su lugar (de "squeeze blank" o "comprimir en blanco").

Para ver el contenido de un archivo con las líneas numeradas y las líneas en blanco comprimidas:

```
cat -ns nombre_archivo.txt
```

1.12. Eliminación de ficheros (rm)

Se pueden eliminar archivos mediante la orden rm. Su sintaxis es la siguiente

```
rm [-opciones] lista_de_ficheros
```

Sus opciones más utilizada son las siguientes:

Opción	Comando	Descripción
-i	rm -i archivo_viejo.txt	Activa el modo interactivo, pidiendo confirmación antes de eliminar cada archivo o directorio
-v o -verbose	rm -v archivo_viejo.txt	Muestra los archivos y directorios a medida que se van borrando. Esto es útil para ver qué está haciendo el comando.
-f	rm -f archivo_viejo.txt	Fuerza el borrado de archivos, incluso los que están protegidos contra escritura, y suprime las confirmaciones. Se debe usar con precaución.
-r o -R	rm -r /home/documents/micasa.txt	Elimina directorios y su contenido de forma recursiva. Es necesario para borrar directorios no vacíos.
-u	mv -u archivo_viejo.txt /ruta/de/destino	Mueve el archivo o directorio solo si el archivo de origen es más reciente que el archivo de destino, o si el archivo de destino no existe.

Además, también se pueden emplear caracteres comodín, como “*” o “?”. Estos caracteres permiten generar patrones de identificación de ficheros. “*” hace referencia a cualquier cadena de caracteres y “?” solo referencia un único carácter cualquiera. El intérprete de comandos, antes de ejecutar la orden donde se incluyan los caracteres comodines, realiza una acción llamada “expansión de comodines”, donde traduce estos comodines a todas las posibles combinaciones de caracteres, según el patrón aportado. De esta manera, se pueden referir varios archivos a la vez.

También es posible borrar archivos de directorios diferentes:

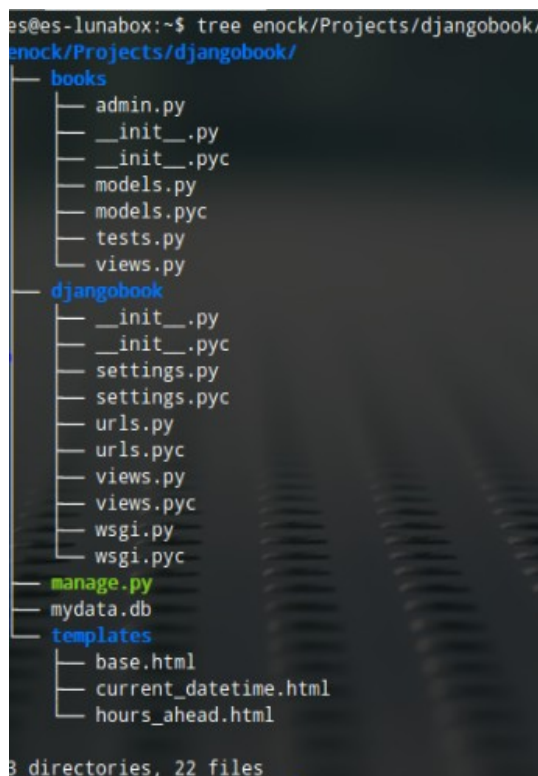
```
rm -i /home/usuario/documentos/archivo1.txt /home/usuario/descargas/archivo2.pdf
```

O borrar todos los archivos txt:

```
rm -i /home/usuario/documentos/*.txt
```

1.13. Mostrar directorios como árbol (tree)

El comando es tree y muestra directorios y archivos en forma de árbol:



```
es@es-lunabox:~$ tree enock/Projects/djangobook/
enock/Projects/djangobook/
├── books
│   ├── admin.py
│   ├── __init__.py
│   ├── __init__.pyc
│   ├── models.py
│   ├── models.pyc
│   ├── tests.py
│   └── views.py
├── djangobook
│   ├── __init__.py
│   ├── __init__.pyc
│   ├── settings.py
│   ├── settings.pyc
│   ├── urls.py
│   ├── urls.pyc
│   ├── views.py
│   ├── views.pyc
│   ├── wsgi.py
│   └── wsgi.pyc
├── manage.py
├── mydata.db
└── templates
    ├── base.html
    ├── current_datetime.html
    └── hours_ahead.html

3 directories, 22 files
```

Algunas opciones interesantes son:

Opción	Comando	Descripción
-d	tree -d	Muestra sólo directorios
-L	tree -L X	Muestra hasta X directorios de profundidad
-f	tree -f	Muestra los archivos con su respectiva ruta
-a	tree -a	Muestra todos los archivos, incluidos los ocultos.
-ugh	tree -ugh	Muestra los ficheros con su respectivo propietario (-u),